Singapore Institute of Technology
Information and Communications Technology Cluster

BAC3004 Capstone Project

Capstone Project Interim Report Submission Form

A. Preamble

This form duly filled and signed must be attached to each Capstone Project Progress Report submitted.

Reporting Period:

(Reporting Period): 5/1/2026 to 10/4/2026

Candidate Particulars:

(Name of Candidate): Lin Zhenming

(Organization): Prudential Assurance Company Singapore

(Project Period): 2/3/2026 to 31/7/2026

Main Academic Supervisor Details:

(Name): Xu Bing Jie

(Email Address): bingjie.xu@singaporetech.edu.sg

(Designation): Assistant Professor

Industry Supervisor Details:

(Name): Chin Wei Shan

(Email Address): wei.shan.chin@prudential.com.sg

(Designation): AI Engineer

(Department / Division): Data & AI Engineering

B. Declaration of Conformity

B.1 Declaration by Industry Supervisor

I hereby acknowledge that I have read and understood the contents of the Capstone Project Progress Report, and deem the contents appropriate for release to the Singapore Institute of Technology for use in the assessment of the candidate.

Signature

Name of Industry Supervisor: Chin Wei Shan
Designation: AI Engineer
Department: Data & AI Engineering
Contact No: Error! Reference source not found.
Email: wei.shan.chin@prudential.com.sg
Date: 6/4/2026

B.2 Declaration by Candidate

I hereby acknowledge that I have engaged and discussed with my Industry Supervisor on the contents of the Capstone Project Progress Report, and have sought approval for the release of the report to the Singapore Institute of Technology.

Signature

Name of Candidate: Lin Zhenming
Contact No: +65 81244215
Email: 2302993@sit.singaporetech.edu.sg
Date: 6/4/2026

END OF FORM

Information and Communications Technology Cluster

Atlas -

Capstone Project Interim Report
(Prudential Assurance Company Singapore)

For the reporting period from 2/3/2026 to 10/4/2026

Lin Zhenming
2302993

Name of Industry Supervisor: Chin Wei Shan

Name of Academic Supervisor: Xu Bing Jie

Submitted as part of the requirement for BAC3004 Capstone Project

Capstone Project Progress Report for the Period of 2/3/2026, to 10/4/2026

Introduction

Background and Motivation

Retail participation in Singapore's financial markets surged during the COVID-19 period, with brokerage account openings reaching record levels across Moomoo, Tiger Brokers, and POEMS. These self-directed investors are typically employed full-time with limited hours for market research, making them receptive to AI-assisted decision support. Yet the tools available to them fall into two unsatisfying categories: fully manual platforms with no AI reasoning, or black-box systems that trade autonomously with no explanation.

The Monetary Authority of Singapore (MAS) reinforces this gap from the regulatory side. Its FEAT principles require AI systems in financial services to be transparent in their decision logic, auditable in their outputs, and supervised by human actors. The MAS Technology Risk Management Guidelines further mandate oversight proportionate to the complexity of the AI application. Singapore's regulatory posture therefore actively penalises black-box trading systems and rewards designs with human-in-the-loop mechanisms.

Gap in Existing Platforms

Commercial AI trading tools fail in one of two ways. Black-box systems like Trade Ideas' Holly AI (USD 178–254/month) generate signals without exposing any reasoning — the user cannot tell why a trade was recommended, let alone whether the AI is making systematic errors. Rule-based platforms like Composer (USD 32/month) and 3Commas go the other direction: they give users full control, but only over manually defined rules. These platforms do not reason independently over market data; they simply automate what the user already knows.

StockHero (USD 30–100/month, 100,000+ users) comes closest as an Alpaca-integrated bot platform for US equities, but still offers no multi-agent reasoning, no reasoning transparency, and no way for the user to configure how much authority the AI has.

No existing retail platform combines three capabilities: a multi-agent reasoning pipeline whose intermediate steps are visible to the user; configurable execution authority over how much independent action the AI may take; and a longitudinal reasoning-trace record that enables the AI's own error patterns to be identified and corrected over time.

Research Question

"How should the boundary of execution authority between human and AI be configured in a retail trading assistant to optimise for both performance and user trust?"

Key Message

Atlas is the first retail trading system that treats the human–AI execution boundary as a measurable, configurable research parameter rather than a fixed architectural decision — enabling controlled comparison of how different authority configurations affect both trading performance and user trust.

Literature Review

Multi-Agent AI Trading Pipelines

TradingAgents (Xiao et al., 2024) proposes a multi-agent LLM framework where specialised agents operate in a coordinated pipeline. The key insight — that dividing analytical labour across specialists produces better-calibrated signals than a monolithic LLM — is adopted by Atlas and extended with the Execution Boundary Controller.

FinMem (Yu et al., 2023) addresses memory in LLM trading agents through a layered system (working, short-term, long-term memory). This directly supports the importance of error-pattern review: agents with access to their own history outperform memoryless agents. Atlas implements this principle through MongoDB reasoning traces and the planned Retrospective Review Agent (Section 5).

AI-Trader (HKUDS, 2024) establishes the evaluation conventions used across the multi-agent trading literature — cumulative return, Sharpe ratio, and maximum drawdown — all three of which Atlas adopts in its backtesting engine. StockClaw (24mlight) replays a full LLM pipeline over frozen historical datasets with strict T−1 data constraints; Atlas independently arrived at the same methodology via an as_of_date parameter that masks future data from every agent. AlphaClaw (熵简科技) introduces Philosophy Skills — named investment philosophies applied as reasoning frameworks at the prompt level — which Atlas adopts as its third experimental axis alongside EBC mode and confidence threshold.

Why LLM-Driven Trading and How to Ensure Reliability

Traditional algorithmic trading systems are fast and deterministic but fundamentally brittle: they cannot reason about a novel earnings surprise or a macroeconomic regime shift unless a programmer has encoded a rule for that exact scenario. LLM-driven systems offer three structural advantages: multi-modal reasoning (jointly processing price action, fundamentals, and sentiment), natural-language explainability (supporting MAS FEAT transparency requirements), and adaptability to novel market conditions without rewriting code.

However, LLM-driven trading introduces reliability and safety concerns that must be addressed architecturally rather than left to human vigilance. Atlas addresses these through five mechanisms: (1) structured output enforcement via Gemini's response_mime_type for schema-validated JSON; (2) a deterministic Risk agent in pure Python that the LLM cannot override (2% portfolio risk per trade, 2:1 reward-to-risk); (3) confidence-gated execution (≥ 65% threshold for autonomous mode); (4) a 5-minute human override window with audit logging; and (5) mandatory reasoning trace persistence before any trade execution.

Human–AI Teaming Theory

The EBC is grounded in two established frameworks. Endsley's situational awareness model (1995) defines three levels — perception, comprehension, and projection — that a human operator must maintain to make sound decisions; Advisory mode is designed to support all three by surfacing the full reasoning trace. Parasuraman's levels of automation taxonomy (2000) distinguishes information-acquisition automation from action-implementation automation; Advisory maps to the former and Autonomous to the latter, with the override window preserving human veto authority. Making this mapping explicit, configurable, and experimentally measurable is Atlas's theoretical contribution to the human–AI teaming literature.

System Design

Architecture Overview

Atlas is a cloud-native web application with three independently deployed layers: a Next.js 16 frontend (Vercel), a FastAPI backend (Render), and a dual-database layer (Supabase PostgreSQL + MongoDB Atlas). Each layer is independently deployable, allowing the LLM-intensive agent pipeline to scale separately from the user-facing frontend. Authentication is handled end-to-end by Clerk via JWT verification with JWKS rotation.

Layer	Technology	Deployment
Frontend	Next.js 16, TypeScript, Tailwind CSS v4	Vercel
Backend	FastAPI, Python 3.11+, LangGraph, Docker	Render
Agents	LangGraph DAG, Gemini 2.5 Flash	Render
Auth	Clerk (JWT + JWKS)	Clerk Cloud
Relational DB	Supabase PostgreSQL (RLS)	Supabase Cloud
Document DB	MongoDB Atlas	MongoDB Cloud
Broker	Alpaca Paper Trading	Alpaca Cloud

Agent Pipeline

The pipeline comprises eight LangGraph nodes executed as a directed acyclic graph (DAG). Three analyst agents run in parallel via fan-out, with a custom operator.or_ reducer merging their outputs. A two-tier model configuration separates "quick" calls (analysts) from "deep" calls (synthesis, decision) via environment variables, balancing latency against reasoning depth.

Node	Role	Type
fetch_data	Retrieves concurrent OHLCV prices, fundamentals, and news via yfinance	Deterministic
Technical Analyst	Analyses price action, indicators, and chart patterns	LLM (Gemini 2.5 Flash)
Fundamental Analyst	Analyses financial statements, ratios, and valuation metrics	LLM (Gemini 2.5 Flash)
Sentiment Analyst	Analyses news headlines and market sentiment	LLM (Gemini 2.5 Flash)
Synthesis	Conducts structured bull/bear debate across merged analyses	LLM (Gemini 2.5 Flash)
Risk	Enforces 2% portfolio risk per trade and 2:1 reward-to-risk ratio	Deterministic (pure Python)
Portfolio Decision	Produces final BUY/SELL/HOLD signal with calibrated confidence (0–1)	LLM (Gemini 2.5 Flash)
save_trace	Persists full reasoning chain to MongoDB before any downstream action	Deterministic

The pipeline supports four investment philosophy overlays applied at the prompt level, shaping how each analyst agent interprets data:

Philosophy	Description	Influence on Agents
Buffett (Value)	Prioritises intrinsic value, margin of safety, and long-term holding	Fundamental Analyst weighs valuation ratios more heavily
Soros (Macro)	Focuses on macroeconomic trends, reflexivity, and regime shifts	Sentiment Analyst prioritises macro news and market momentum
Lynch (Growth)	Targets undervalued growth stocks with strong earnings trajectories	Fundamental Analyst emphasises earnings growth and PEG ratio
Balanced	Equal weighting across all analytical dimensions	No bias — default configuration

The Risk node is deliberately implemented without any LLM involvement — it enforces hard position-sizing constraints that no other agent in the pipeline can override. No trade can execute without save_trace completing first, ensuring a complete audit trail for every pipeline run.

Execution Boundary Controller

The EBC sits between Portfolio Decision and the broker adapter. The same pipeline, data, and risk rules produce identical reasoning regardless of mode — only execution authority changes. This isolation makes the EBC a valid independent variable for the user study.

Mode	Behaviour	Threshold	Override
Advisory	Signals queue for human review. No automated execution.	N/A	N/A
Autonomous	AI executes when confidence ≥ threshold. Human has override window.	≥ 65%	5 min

Backtesting Engine

The backtesting engine replays the real Gemini pipeline over historical date ranges rather than substituting a rule-based simulation — a deliberate methodological choice to measure actual AI decision quality. The as_of_date parameter constrains price and fundamental data to the simulated date, preventing look-ahead bias at the data layer. The virtual portfolio starts at USD 10,000 with USD 1,000 notional per trade, executing at the next trading day's open. Advisory mode produces signals only (total_trades = 0), providing a clean no-execution baseline for comparison.

Known limitation: yfinance's news endpoint returns current headlines regardless of as_of_date, introducing potential look-ahead bias in the Sentiment Analyst's inputs. A fix via the Alpaca News API is planned for the final submission.

Database Architecture

Supabase PostgreSQL stores structured application data across 7 tables (profiles, portfolios, positions, trades, override_log, backtest_jobs, broker_connections) with Row-Level Security enforced on every table, managed via 10 incremental migrations.

MongoDB Atlas stores 2 collections (reasoning_traces, backtest_results). The document model accommodates variable-depth agent outputs — each analyst may produce different sub-fields depending on data availability — without requiring schema migrations.

Implementation

Build Progress

The system is ahead of schedule. Broker integration (originally Phase 4) and backtesting (Phase 3) were both completed within Phase 2. All 11 originally scoped features are live with no stubs.

Component	Status
Agent pipeline (5 agents, parallel LangGraph)	Live
Execution Boundary Controller (2 modes)	Live
Auth (Clerk JWT end-to-end)	Live
Broker (Alpaca paper trading, per-user)	Live
Override window (cancel + audit log)	Live
Signal approval/rejection (idempotent)	Live
Backtesting engine (async, real Gemini)	Live
Pricing page (Free/Pro/Max)	Live
Scheduled pipeline runs (9:30 AM ET)	Live
RBAC (User/Admin/SuperAdmin)	Live
Investment philosophy overlay	Live

API Surface

The backend exposes 12 authenticated API endpoints: health check; pipeline execution; signal fetch, approve, reject; portfolio data and equity curve; trade history and override; backtest create, list, get, delete. All routes are versioned under /v1/ and require Clerk JWT verification. The API-first architecture means the dashboard is one client among many — enabling future mobile and developer API access.

Key Technical Decisions

The following table summarises key technical decisions and their rationale. Each choice was driven by a specific architectural requirement rather than framework popularity.

Decision	Rationale
Gemini 2.5 Flash	Cost-effective, supports schema-enforced JSON output
LangGraph	Native parallel execution, explicit state, conditional routing
Deterministic Risk agent	Pure Python safety floor the LLM cannot override
Real LLM replay in backtesting	Tests actual AI quality, not a proxy
Two EBC modes (not three)	Conditional collapsed into Advisory — both require human action
Mandatory trace persistence	save_trace is a graph node, not optional middleware

Error Pattern Recording and Retrospective Review

Feedback from trading professionals during early demos, combined with discussions with AI practitioners, converged on a critical insight: it is essential for AI agents to record and review their own past error patterns. This aligns with FinMem's finding that memory-augmented agents outperform memoryless agents, and has been elevated from a planned enhancement to a core architectural priority.

Atlas's reasoning trace architecture provides the data foundation. Every pipeline run persists a structured document to MongoDB containing: each analyst's signal, indicators, and reasoning; the synthesis bull/bear debate; risk parameters; the final decision with confidence; and execution outcome. When combined with subsequent price data, each trace becomes a labelled datapoint — correct or incorrect.

Three levels of error-pattern analysis are planned:

- Level 1 — Dashboard Error Analytics: Aggregate traces with outcome data to surface directional accuracy by ticker, by analyst, and by confidence band. Reveals patterns such as "Sentiment Analyst is consistently wrong on TSLA during earnings weeks."

- Level 2 — Retrospective Review Agent: A new pipeline node that queries MongoDB for recent traces on the same ticker and summarises the AI's track record before Synthesis. The prompt asks: "What error patterns have occurred in recent decisions on this ticker?"

- Level 3 — Confidence Recalibration: Use accumulated trace-vs-outcome data to empirically recalibrate the EBC's confidence threshold. If 70% confidence signals only win 55% of the time, the threshold should be raised. This closes the feedback loop between past performance and future execution authority.

The key architectural decision enabling all three levels is that trace persistence is a mandatory graph node (save_trace), not optional middleware. Every run — live or backtest — produces a trace regardless of execution.

Technical Challenges

Parallel Agent Execution

LangGraph's fan-out requires careful state management. The three analyst agents write concurrently to a shared AgentState TypedDict; a custom Annotated dict reducer using operator.or_ merges their parallel outputs into a single state object. Early testing revealed that malformed JSON from one agent could silently corrupt the merged state, so debugging required trace-level logging to isolate which agent produced the faulty output. This challenge led directly to the structured JSON enforcement (via Gemini's response_mime_type) now applied to all LLM calls.

Three-Way Authentication

Clerk, Supabase, and FastAPI each impose different JWT expectations — different signing algorithms, audiences, and claim structures. The solution: Clerk issues JWTs using a custom atlas-supabase template configured with HS256, the Supabase JWT secret, and audience "authenticated". The backend independently verifies tokens via JWKS and extracts the sub claim as user ID. A frontend AuthSync component upserts profiles on every sign-in, keeping all three systems synchronised without manual account provisioning.

Idempotent Trade Execution

The trade approval endpoint must be idempotent — a network retry or duplicate click must not place a second order with the broker. Before calling Alpaca, the solution checks MongoDB's execution_executed flag on the reasoning trace; if already executed, it returns the existing order ID rather than placing a duplicate.

Backtesting Look-Ahead Bias

An as_of_date parameter threads through every data fetcher to prevent the AI from "seeing the future." Price and fundamental data are truncated at the simulated date, ensuring each agent reasons only on information that was available at that point in time. However, yfinance's news endpoint ignores date parameters — a documented limitation that introduces potential look-ahead bias in the Sentiment Analyst. A fix via the Alpaca News API is planned for the final submission.

Evaluation Framework

Quantitative Metrics

The following metrics are computed automatically from backtesting results and live trading data, enabling objective comparison across EBC modes and configurations:

Metric	Description
Cumulative Return	Total return as % of starting capital
Sharpe Ratio	Annualised excess return per unit volatility (risk-free = 0%)
Maximum Drawdown	Largest peak-to-trough decline
Win Rate	Proportion of closed trades with positive return
Signal-to-Execution Rate	Signals resulting in trades (Advisory: 0%, Autonomous: ~100%)
Directional Accuracy	Signals whose direction matched subsequent price movement

Qualitative Metrics (UAT — Phase 5)

A user acceptance test in Phase 5 will collect qualitative data per EBC mode: user confidence in the AI's recommendations, decision regret (whether users wished they had overridden or followed the AI), reasoning clarity (whether the trace was understandable), mode preference, and override satisfaction. These metrics capture the trust dimension that quantitative performance alone cannot measure.

Composite Optimal Boundary Score

A composite Optimal Boundary Score (OBS) combines both dimensions into a single numerical comparison:

$$OBS = \alpha \cdot Sharpe + \beta \cdot (1 - |MaxDrawdown|) + \gamma \cdot TrustScore + \delta \cdot ExecutionEfficiency$$

Weighting coefficients (α, β, γ, δ) will be determined empirically from UAT results. The OBS enables direct comparison across EBC modes — a higher score indicates a configuration that better balances trading performance with user trust.

Three Experimental Axes

The evaluation is structured around three experimental axes:

Axis	Variable	Conditions
1 (Primary)	EBC Mode	Advisory vs. Autonomous
2	Orchestration	v2 parallel fan-out vs. v3 Adaptive Conductor
3	Philosophy Skill	Buffett / Soros / Lynch / Balanced

Axis 1 is the primary research variable tied to the research question. Axes 2 and 3 are secondary variables that test whether pipeline architecture and investment philosophy influence the optimal boundary configuration.

Early Results

System Status

All core components are deployed to UAT (frontend: atlas-broker-frontend-uat.vercel.app; backend: atlas-broker-backend-uat.onrender.com). End-to-end flows have been validated: onboarding, pipeline execution, signal approval/rejection, autonomous execution with override, and scheduled daily runs at 9:30 AM ET.

Backtesting Status

The backtesting engine is functional and initial batches are in progress: 5 tickers (AAPL, MSFT, TSLA, NVDA, META) × 2 modes × 60-day window = 10 jobs, each involving approximately 300 real Gemini API calls. Completed results with equity curves and Sharpe ratios will be reported in the final submission. The sentiment look-ahead bias fix remains the key outstanding item before results can be considered fully reliable.

Preliminary Observations

- The three-analyst fan-out consistently produces richer context than any single analyst. Cases of technical/fundamental disagreement are particularly informative — the synthesis debate surfaces tension explicitly, a structural advantage over monolithic approaches.

- Sentiment signals show the most day-to-day variance, validating the look-ahead bias fix as high priority and suggesting the Retrospective Review Agent should weight sentiment error patterns heavily.

- The 65% confidence threshold appears well-calibrated: sub-threshold signals show weaker analyst consensus. Rigorous validation via full backtest data is needed before this can be confirmed statistically.

- Reasoning trace clarity is high. Trading professionals in early demos found the structured traces comprehensible and useful — their strongest feedback was that the system should learn from its own mistakes over time, directly motivating the error-pattern architecture in Section 5.

Knowledge Applied

Classroom Knowledge

The following modules from the SIT curriculum were directly applied in this project:

- Programming / Software Engineering: Python (FastAPI, LangGraph) and TypeScript (Next.js 16) form the backend and frontend respectively. API-first design principles and test suites (Jest + pytest) ensure maintainability.

- Database Systems: The dual-database architecture applies relational theory (PostgreSQL with ACID guarantees and Row-Level Security) alongside document modelling (MongoDB for flexible agent outputs). Ten incremental schema migrations manage evolution.

- Web Development: Full-stack REST API design, JWT authentication, CORS configuration, Tailwind CSS v4, and dark/light theming.

- Fintech: Trade lifecycles, risk management, portfolio metrics, and broker API integration via Alpaca.

Beyond-Classroom Knowledge

The project required significant self-directed learning in areas not covered by the curriculum:

- Multi-agent systems: LangGraph DAG orchestration, parallel fan-out/fan-in execution, and custom state reducers for merging concurrent agent outputs.

- LLM structured output: Gemini schema-enforced JSON via response_mime_type, two-tier model configuration, and financial prompt engineering tailored to each analyst role.

- Human–AI teaming: Parasuraman's levels of automation and Endsley's situational awareness model, applied to design the EBC's two modes.

- Explainable AI and error-pattern learning: Reasoning trace architecture serving as both audit trail and longitudinal learning dataset for confidence recalibration.

- Cloud-native deployment: Docker containerisation, uv for dependency management, Vercel/Render deployment, webhook-driven sync, and role-based access control.

Remaining Work

Critical Path (required for final report results):

Sentiment Look-Ahead Bias Fix: Replace yfinance news with Alpaca News API date-range parameters. This is the highest-priority item — backtesting results cannot be considered fully reliable until sentiment data respects the as_of_date constraint.

Cross-Mode Statistical Analysis: Complete all backtest batches and conduct rigorous cross-mode, cross-ticker comparison with statistical significance testing.

Core Enhancements:

Error-Pattern Dashboard (Level 1): Surface directional accuracy by ticker, analyst, and confidence band — the first layer of the error-pattern architecture described in Section 5.

Retrospective Review Agent (Level 2): A new pipeline node that queries past traces on the same ticker to inform current analysis, closing the feedback loop between historical performance and live decisions.

v3 Adaptive Conductor: A meta-agent that selectively spawns analysts based on market context, replacing the fixed three-analyst fan-out with context-aware orchestration.

Infrastructure and Evaluation:

Circuit Breaker: Pause auto-execution on consecutive losses or high drawdown, adding a runtime safety mechanism beyond the static risk constraints.

Stripe Integration: Wire pricing tiers to billing and enforce Free tier API limits at the backend level.

User Acceptance Testing: Conduct UAT with supervisors and retail investors to collect the qualitative metrics defined in the Evaluation Framework.

References

Endsley, M. R. (1995). Toward a theory of situation awareness in dynamic systems. Human Factors, 37(1), 32–64.

Monetary Authority of Singapore. (2021). Principles to Promote Fairness, Ethics, Accountability and Transparency (FEAT) in the Use of Artificial Intelligence and Data Analytics in Singapore's Financial Sector. MAS.

Monetary Authority of Singapore. (2021). Technology Risk Management Guidelines. MAS.

Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A model for types and levels of human interaction with automation. IEEE Trans. SMC—Part A, 30(3), 286–297.

Xiao, Y., et al. (2024). TradingAgents: Multi-agents LLM financial trading framework. arXiv:2412.20138.

Yu, S., et al. (2023). FinMem: A performance-enhanced LLM trading agent with layered memory. arXiv:2311.13743.

Zhang, J., et al. (2024). AI-Trader: LLM-based framework for automated trading. arXiv:2512.10971.

熵简科技 (AlphaClaw). (2024). Philosophy-driven multi-agent trading. https://alphaengine.top

24mlight (StockClaw). (2024). Root-agent framework for AI-assisted equity trading. https://github.com/24mlight/StockClaw

IOSCO. (2025). The Use of Artificial Intelligence and Machine Learning by Market Intermediaries and Asset Managers. CR/01/2025.

Trade Ideas. (2025). Holly AI. https://www.trade-ideas.com

Composer Technologies. (2025). Composer. https://www.composer.trade

3Commas. (2025). 3Commas. https://3commas.io

StockHero. (2025). StockHero. https://www.stockhero.ai

Appendices

The following appendices provide supplementary technical detail and are excluded from the word count.

Appendix A: Database Schema

Supabase

Table	Key Columns	Purpose
profiles	id (Clerk ID), email, boundary_mode, tier, role, philosophy	User metadata
portfolios	user_id, cash (default $100K)	One per user
positions	portfolio_id, ticker, shares, avg_cost	Open positions
trades	user_id, ticker, action, price, status, signal_id, order_id	Execution record
override_log	trade_id, reason, broker_cancel_success	Cancellation audit
backtest_jobs	user_id, status, tickers, ebc_mode, metrics	Job metadata
broker_connections	user_id, api_key, environment	Per-user credentials

MongoDB

Collection	Key Fields	Purpose
reasoning_traces	user_id, ticker, analysts, synthesis, risk, decision, confidence, timestamp	Full pipeline audit trail
backtest_results	job_id, ticker, date, decision, confidence, portfolio_snapshot	Backtest outcome data

Appendix B: Competitive Positioning

Platform	Price	Multi-Agent	Transparency	Configurable Execution Authority	Error Learning
Atlas	Free/$49/$149	Yes	Full trace	Yes	Yes (planned)
Trade Ideas	$178–254	No	None	No	No
Composer	$32	No	Rules only	No	No
StockHero	$30–100	No	None	No	No
StockClaw	Free (OSS)	Yes	Partial	No	No